SIMATS ENGINEERING

ASSESSMENT TOOL 2

Annexure A

Constraint-Based Problem Solving

COURSE INFORMATION

Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome Covered	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3)
Assessment Tool Weightage	35%
Questions / Total Marks	3 questions / 30 marks
Duration	1 Hour

SUBMISSION DETAILS

Student Name	G. Venkateswarlu
Register Number	192412607
Course Faculty	Dr K Rajagopal
Date of Submission	14 October 2026

CODE OF CONDUCT

I, **G. Venkateswarlu (Reg. No. 192412607)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Venkateswarlu **Date:** 14 October 2026

Annexure A

Constraint-Based Problem Solving

Question 1: Doctor Shift Scheduling (Backtracking Search)

Problem. Three doctors (D1, D2, D3) must be assigned to three shifts (Morning, Afternoon, Night) subject to five constraints. Backtracking search is applied with all intermediate steps and constraint checks, and the final valid schedule is stated.

1.1 Formulation as a Constraint Satisfaction Problem

Element	Definition
Variables	D1, D2, D3 (the shift given to each doctor)
Domain	{Morning (M), Afternoon (A), Night (N)}, ordered M < A < N
Constraint C1	D1 ≠ Night
Constraint C2	Shift(D2) < Shift(D3), so D2 works before D3
Constraint C3	Only one doctor per shift (all three values are different)
Constraint C4	D3 ≠ Morning
Constraint C5	Every doctor is assigned exactly one shift (the assignment must be complete)

1.2 Method

Backtracking search is a depth-first search that assigns one variable at a time, checks every constraint that can already be tested after each assignment, and returns to the previous variable (backtracks) as soon as a value breaks a constraint. The variable order is D1, D2, D3 and the value order is Morning, Afternoon, Night. A constraint that involves an unassigned variable (for example C2 while D3 is unassigned) is checked later, when all its variables have values.

1.3 Step-by-Step Search with Constraint Checks

Step	Assignment tried	Constraint checks	Decision
1	D1 = Morning	C1 satisfied (not Night); C3 satisfied (no other doctor assigned)	Accept. Partial: {D1 = M}
2	D2 = Morning	C3 violated (Morning already given to D1)	Reject; try next value
3	D2 = Afternoon	C3 satisfied; C2 cannot be tested yet (D3 unassigned)	Accept. Partial: {D1 = M, D2 = A}
4	D3 = Morning	C3 violated (Morning taken); C4 violated (D3 cannot work Morning)	Reject; try next value
5	D3 = Afternoon	C3 violated (Afternoon taken by D2); C2 violated (A < A is false)	Reject; try next value
6	D3 = Night	C3 satisfied; C4 satisfied; C2 satisfied (A < N); C1 satisfied; C5 satisfied (all three assigned)	Accept. Complete assignment: goal reached

1.4 Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Final schedule returned by backtracking search

1.5 Verification and Completeness Check

Each constraint is checked against the final schedule (D1 = Morning, D2 = Afternoon, D3 = Night):

Rule	Requirement	Result for the schedule
C1	D1 cannot work Night	D1 works Morning: satisfied
C2	D2 works before D3	Afternoon is before Night: satisfied
C3	One doctor per shift	Three different shifts: satisfied
C4	D3 cannot work Morning	D3 works Night: satisfied
C5	Each doctor has exactly one shift	Each of D1, D2, D3 has one shift: satisfied

Continuing the search after the first solution shows the backtracking explicitly. D3 has no further values, so the search returns to D2 and tries D2 = Night, which fails for every D3 value (Morning violates C4, Afternoon violates C2, Night violates C3). It then returns to D1 and tries D1 = Afternoon, which leads to D2 = Morning and D3 = Night, a second valid schedule. D1 = Night is rejected immediately by C1. All six possible assignments are summarised below.

D1	D2	D3	Outcome
Morning	Afternoon	Night	Valid (first solution found, reported as the answer)
Morning	Night	Afternoon	Invalid: C2 violated (Night is after Afternoon)
Afternoon	Morning	Night	Valid (second solution)
Afternoon	Night	Morning	Invalid: C2 and C4 violated
Night	Morning	Afternoon	Invalid: C1 violated
Night	Afternoon	Morning	Invalid: C1, C2 and C4 violated

Conclusion. Backtracking search returns the schedule **D1 = Morning, D2 = Afternoon, D3 = Night**. The only other valid schedule is D1 = Afternoon, D2 = Morning, D3 = Night.

Question 2: Robot Navigation on a 5×5 Grid (BFS and Priority Queue)

Problem. A robot moves from Start (S) to Goal (G) on a 5×5 grid. It may move Up, Down, Left or Right, each move costs 1, and obstacles cannot be crossed. The task asks for step-by-step node expansion and priority queue updates using BFS with the Manhattan distance heuristic, and the optimal path and cost.

2.1 Assumed Environment

The question paper does not reproduce the grid, so the following layout is assumed. Cells are written as (row, column), counted from 0 at the top-left corner.

	c0	c1	c2	c3	c4
r0	S				
r1					
r2					
r3					
r4					G

Assumed 5×5 grid: S = start (0,0), G = goal (4,4), dark cells = obstacles at (1,1), (1,3), (2,1) and (3,3)

2.2 Method and Justification

Breadth-First Search (BFS) keeps a FIFO queue and expands nodes level by level. Because every move costs 1, the first time BFS reaches G it has found a cheapest path, so BFS is optimal here. BFS itself ignores heuristics, so the Manhattan distance is applied in the second part, where the queue becomes a **priority queue** ordered by f(n) = g(n) + h(n) (the A* form of best-first search). Both searches use the neighbour order Up, Down, Left, Right, and ties in the priority queue are broken by smaller h, then by first-in first-out order.

The heuristic h(n) = |row(n) − 4| + |column(n) − 4| never overestimates the remaining cost (each move changes the distance by at most 1), so it is admissible and consistent, which guarantees that the first path returned by the priority-queue search is optimal.

	c0	c1	c2	c3	c4
r0	S 8	7	6	5	4
r1	7		5		3
r2	6		4	3	2
r3	5	4	3		1

r4	**4**	**3**	**2**	**1**	**G 0**

Manhattan distance h(n) = |row − 4| + |column − 4| for each free cell (dark cells are obstacles)

2.3 Part A: BFS Node Expansion and Queue Updates

The goal test is applied when a node is generated. Each row shows the node removed from the front of the queue, the new nodes added to the back, and the queue afterwards (front on the left).

Step	Node expanded	Nodes added to queue	Queue after step (front to back)
1	(0,0)	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	(2,0)	(0,1), (2,0)
3	(0,1)	(0,2)	(2,0), (0,2)
4	(2,0)	(3,0)	(0,2), (3,0)
5	(0,2)	(1,2), (0,3)	(3,0), (1,2), (0,3)
6	(3,0)	(4,0), (3,1)	(1,2), (0,3), (4,0), (3,1)
7	(1,2)	(2,2)	(0,3), (4,0), (3,1), (2,2)
8	(0,3)	(0,4)	(4,0), (3,1), (2,2), (0,4)
9	(4,0)	(4,1)	(3,1), (2,2), (0,4), (4,1)
10	(3,1)	(3,2)	(2,2), (0,4), (4,1), (3,2)
11	(2,2)	(2,3)	(0,4), (4,1), (3,2), (2,3)
12	(0,4)	(1,4)	(4,1), (3,2), (2,3), (1,4)
13	(4,1)	(4,2)	(3,2), (2,3), (1,4), (4,2)
14	(3,2)	none (all neighbours already visited)	(2,3), (1,4), (4,2)
15	(2,3)	(2,4)	(1,4), (4,2), (2,4)
16	(1,4)	none (all neighbours already visited)	(4,2), (2,4)
17	(4,2)	(4,3)	(2,4), (4,3)
18	(2,4)	(3,4)	(4,3), (3,4)
19	(4,3)	(4,4) (goal generated)	(3,4), (4,4)

BFS generates the goal G at step 19 after expanding 19 nodes. Following the parent pointers back from G gives the path (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4).

2.4 Part B: Priority Queue Updates with the Manhattan Heuristic

Each row shows the node removed from the priority queue (lowest f), the nodes generated with their g, h and f values, and the priority queue afterwards in the form node:f, with the next node to be removed on the left.

Step	Node removed (g, h, f)	Nodes generated (g, h, f)	Priority queue after step
1	(0,0) g=0, h=8, f=8	(1,0) g=1, h=7, f=8; (0,1) g=1, h=7, f=8	(1,0):8, (0,1):8
2	(1,0) g=1, h=7, f=8	(2,0) g=2, h=6, f=8	(2,0):8, (0,1):8

Step	Node removed (g, h, f)	Nodes generated (g, h, f)	Priority queue after step
3	(2,0) g=2, h=6, f=8	(3,0) g=3, h=5, f=8	(3,0):8, (0,1):8
4	(3,0) g=3, h=5, f=8	(4,0) g=4, h=4, f=8; (3,1) g=4, h=4, f=8	(4,0):8, (3,1):8, (0,1):8
5	(4,0) g=4, h=4, f=8	(4,1) g=5, h=3, f=8	(4,1):8, (3,1):8, (0,1):8
6	(4,1) g=5, h=3, f=8	(4,2) g=6, h=2, f=8	(4,2):8, (3,1):8, (0,1):8
7	(4,2) g=6, h=2, f=8	(3,2) g=7, h=3, f=10; (4,3) g=7, h=1, f=8	(4,3):8, (3,1):8, (0,1):8, (3,2):10
8	(4,3) g=7, h=1, f=8	(4,4) g=8, h=0, f=8	(4,4):8, (3,1):8, (0,1):8, (3,2):10
9	(4,4) g=8, h=0, f=8	Goal test satisfied: search stops	Optimal path found

Every node on the path has f = 8, the Manhattan lower bound, so the priority queue is led straight down the left column and along the bottom row. The goal is removed at step 9, after only 8 expansions.

2.5 Optimal Path and Cost

	c0	c1	c2	c3	c4
r0	S				
r1	1				
r2	2				
r3	3				
r4	4	5	6	7	G

Optimal path: cells are numbered in the order visited (S = 0, G = 8)

Result	Value
Optimal path	(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)
Directions	Down, Down, Down, Down, Right, Right, Right, Right
Optimal cost	8 (eight moves at cost 1 each)
Nodes expanded	BFS: 19; priority queue with Manhattan heuristic: 8

Interpretation. The Manhattan distance from S to G is 8 and the path found has cost 8, so no path can be cheaper and the result is optimal. Other cost-8 paths exist (for example along the top row and right column); the neighbour order and tie-breaking rule select the left-column route. BFS and the heuristic search return the same optimal cost, but the heuristic reduces the work from 19 expansions to 8 because the priority queue favours nodes closer to G.

Question 3: Autonomous Rescue Robot (Online Search)

Problem. A rescue robot must travel from S to a survivor at G in a partially observable building grid. It senses only adjacent cells, some cells are blocked, risky zones add a penalty, and the robot must update its knowledge as it moves.

3.1 Assumed Environment

No grid is supplied, so a 5×5 building is assumed to demonstrate the approach. The robot knows its own position and the position of G but not which cells are blocked or risky until it senses them. Entering an ordinary cell costs 1; entering a risky cell costs 1 + 2 = 3.

	c0	c1	c2	c3	c4
r0	S				
r1					
r2			R		
r3				R	
r4		R			G

Assumed building layout (hidden from the robot at the start): S = start, G = survivor, dark = blocked, R = risky zone

Task (a): Analysis of the Constraints

The constraints are listed in order of priority: hard feasibility rules first, then the information limits, then the cost objective.

Constraint	Effect on the robot's decision-making
Moves: up, down, left, right	At most four successors per cell and no diagonal shortcuts, so every path is a sequence of grid steps and the search space stays small.
Blocked cells	A hard constraint: such cells are never entered. Discovering one can make the current plan invalid, so the robot must be able to replan, and it must report failure if every route to G is closed.
Limited sensing (adjacent cells only)	The environment is partially observable, so the robot cannot compute a guaranteed route in advance. It stores what it has sensed (a map or belief state), plans with an assumption for unknown cells, and decides again after every percept.
Step cost 1, risky entry +2	Costs are unequal, so BFS and DFS cannot minimise cost; a cost-sensitive search such as Uniform Cost Search is needed. Entering a risky cell costs 2 more than an ordinary cell, and a detour on a grid adds an even number of extra moves, so a detour of 2 extra moves only ties with the risky cell and a detour of 4 or more extra moves is strictly worse. Ties are broken in favour of the safe route.

Constraint	Effect on the robot's decision-making
Minimise total cost with safe navigation	The objective combines optimality and safety: blocked cells are forbidden, risky cells are used only when unavoidable or clearly cheaper, and among equal-cost routes the safer one is chosen.
Dynamic (online) knowledge update	Planning and acting are interleaved: the robot senses, updates its map, replans, and executes one move at a time. Any move already made cannot be undone, so early optimistic assumptions can cost extra steps; the plan is only optimal for the knowledge held at that moment.

Task (b): Solution Approach

The chosen strategy is an **online search agent that repeatedly applies Uniform Cost Search (UCS) on its current map** (sense, plan, move one step, repeat). Cells the robot has not yet sensed are assumed to be free with cost 1 (an optimistic assumption), so the agent keeps moving toward G until a percept proves otherwise.

1. **Initialise.** Knowledge = {S is free}; position = S; total cost = 0.
2. **Sense.** Observe the four adjacent cells and record each as free, blocked or risky.
3. **Plan.** Run UCS from the current position to G on the known map. A free cell costs 1, a risky cell costs 3, a blocked cell is not allowed, and an unknown cell is treated as free.
4. **Act.** Execute only the first move of the cheapest plan and add its cost to the total.
5. **Repeat.** Return to step 2 from the new cell. If a newly sensed blocked cell breaks the plan, the next UCS run automatically finds a new route. Stop when the robot reaches G; if UCS finds no path, report that the survivor is unreachable.

Pseudocode

```
function ONLINE-UCS-RESCUE-AGENT(S, G)
    knowledge ← {S : free};  position ← S;  totalCost ← 0
    while position ≠ G do
        for each adjacent cell x of position do
            knowledge[x] ← SENSE(x)          // free, blocked or risky
        plan ← UCS(position, G, knowledge)  // free = 1, risky = 3, blocked = forbidden, unknown = 1
        if plan = failure then return "Survivor unreachable"
        next ← FIRST-STEP(plan)
        MOVE(next);  totalCost ← totalCost + COST(next)
        position ← next
    return totalCost
```

Demonstration on the assumed building

The table shows each cycle of the agent. The estimated cost in square brackets is the cost of the cheapest remaining route according to the robot's knowledge at that moment.

Step	Position	Newly sensed	Plan to G after update [cost]	Move	Cost so far
1	(0,0)	(1,0) free; (0,1) free	(1,0), (2,0), (3,0), (4,0), (4,1), (4,2), (4,3), (4,4) [8]	Down to (1,0)	1

Step	Position	Newly sensed	Plan to G after update [cost]	Move	Cost so far
2	(1,0)	(2,0) free; (1,1) BLOCKED	(2,0), (3,0), (4,0), (4,1), (4,2), (4,3), (4,4) [7]	Down to (2,0)	2
3	(2,0)	(3,0) BLOCKED; (2,1) free	(2,1), (3,1), (4,1), (4,2), (4,3), (4,4) [6]	Right to (2,1)	3
4	(2,1)	(3,1) free; (2,2) RISKY	(3,1), (4,1), (4,2), (4,3), (4,4) [5]	Down to (3,1)	4
5	(3,1)	(4,1) RISKY; (3,2) free	(3,2), (4,2), (4,3), (4,4) [4]	Right to (3,2)	5
6	(3,2)	(4,2) free; (3,3) RISKY	(4,2), (4,3), (4,4) [3]	Down to (4,2)	6
7	(4,2)	(4,3) free	(4,3), (4,4) [2]	Right to (4,3)	7
8	(4,3)	(4,4) free	(4,4) [1]	Right to (4,4)	8

	c0	c1	c2	c3	c4
r0	S				
r1	1				
r2	2	3	R		
r3		4	5	R	
r4		R	6	7	G

Path actually travelled by the robot (cells numbered in order visited); risky zones were avoided

At step 3 the robot senses that (3,0) is blocked, so the straight route down the left column is no longer valid and UCS immediately returns a new plan through (2,1). The risky cells (2,2), (4,1) and (3,3) are sensed but never entered, because every route through them costs more than the safe route. The robot reaches G with a total cost of 8, which equals the cost 8 of the best route that could be found by an agent that knew the whole map in advance.

Task (c): Application of AI Concepts

Intelligent agent

The robot is a **model-based, goal-based agent with cost (utility) evaluation**: it maintains an internal map built from percepts, has the goal of reaching the survivor, and compares actions by their total cost. Its PEAS description is given below.

PEAS element	Description
Performance measure	Reach G with minimum total cost, never enter blocked cells, avoid risky zones unless necessary, keep planning time small
Environment	Building grid of rooms with blocked cells, risky zones and one survivor
Actuators	Drive mechanism for moving up, down, left and right; stop
Sensors	Proximity or camera sensors for the four adjacent cells; position (odometry) sensor

Rationality

A rational agent chooses, at every step, the action expected to maximise its performance measure given its percept history and built-in knowledge. Rationality is not omniscience: the robot cannot know unseen cells, so it acts on the best available knowledge and corrects itself when a percept contradicts it. In the demonstration it always executes the first move of the cheapest known route, treats information gathering (sensing) as part of every step, replans when (3,0) is found blocked, and refuses risky cells whose penalty exceeds any detour cost. Its final cost equals the best achievable cost, which shows rational behaviour.

Environment type

Property	Classification	Reason
Observability	Partially observable	Only the adjacent cells are sensed
Agents	Single agent	The robot acts alone; survivors and obstacles do not act
Determinism	Deterministic (assumed)	A chosen move leads to the intended cell
Episodic / sequential	Sequential	Each move affects later positions and the total cost
Static / dynamic	Static layout, changing knowledge	The cells do not change, but the robot's knowledge of them does
Discrete / continuous	Discrete	Rooms form a grid with a finite set of moves
Known / unknown	Unknown (partially known)	The map must be discovered while moving

Task (d): Justification of the Chosen Approach

Strategy	Handles unequal costs	Optimal on known map	Works with online sensing	Verdict
BFS	No	Only for equal costs	Yes, with replanning	Ignores the risky-zone penalty
DFS	No	No	Yes	May wander and give costly paths
Greedy best-first	No	No	Yes	Can walk into risky zones
Offline planning	Yes	Yes	No	Impossible: the map is unknown
Online UCS with replanning	Yes	Yes	Yes	Best fit for all constraints

- **Cost-sensitive and optimal.** UCS expands the cheapest paths first, so the plan is optimal for the robot's current knowledge and the risky-zone penalty is built into the cost.
- **Handles partial observability.** Replanning after every percept lets the robot react when a blocked or risky cell appears, which no fixed offline plan can do.

- **Safe and complete.** Blocked cells are excluded from the search, risky cells are used only when they save more than they cost, and an empty UCS result tells the robot that the survivor is unreachable.
- **Efficient enough.** The building has few cells, so rerunning UCS after each step is cheap. A Manhattan-distance heuristic (A*) could speed up each run, but it is not needed to meet the constraints, and UCS keeps the solution within the uninformed search methods of CO1.

Conclusion. An online search agent that senses, updates its map and replans with Uniform Cost Search satisfies every stated constraint and reaches the survivor at the minimum possible cost.